

Tier 1: Core Fundamentals & Locators

Assignment 2 — Synchronization & Explicit Waits

Name	Ashmita Saha
Enrollment No.	12023002028118
Department	CSE (AI & ML)
Course	Python for Automation
Academic Session	2023-2027
Submission Date	11 Sept, 2026

Module Name and Experiment Title

Module: Tier 1 – Core Fundamentals & Locators

Experiment Title: Assignment 2 – Synchronization & Explicit Waits, i.e. waiting for dynamic content to actually show up before reading it, instead of guessing with a sleep timer

Problem Statement

A lot of pages don't load everything at once — some text or element only shows up after a button click, an AJAX call, or just a delay built into the page itself. If a script tries to read that element too early, it either grabs nothing or throws an error because the element isn't there yet. For this task I used the dynamic loading demo page (the one with a 'Start' button that reveals text after about 5 seconds) and had to make sure the script waits for that text to actually appear before trying to read it, without using `time.sleep()` as a workaround.

Objective

- Understand why fixed delays like `time.sleep()` are a bad way to handle dynamic content, and use `WebDriverWait` instead.
- Get used to `expected_conditions` (EC) — specifically waiting for an element to be clickable and for another to become visible.
- Automate clicking the 'Start' button and then reading the text that only appears afterward.
- Confirm the extracted text matches what's expected, using a simple assertion.

Tools, Software, and Concepts Used

- **Language:** Python 3
- **Library:** Selenium WebDriver
- **Browser/Driver:** Chrome, run through ChromeDriver
- **Site used for testing:** the-internet.herokuapp.com/dynamic_loading/1
- **Concepts I used:** `WebDriverWait` combined with `expected_conditions` (EC), specifically `element_to_be_clickable` and `visibility_of_element_located`, along with a plain `assert` to check the final text and `driver.quit()` to close things out.

Source Code / Implementation Steps

1. Import `webdriver`, `By`, `WebDriverWait` and `expected_conditions` (as `EC`) from Selenium.
2. Launch Chrome and open the dynamic loading demo page, then maximize the window.
3. Set up a `WebDriverWait` with a 10 second timeout so the script doesn't wait forever.
4. Wait until the 'Start' button becomes clickable (found via `XPATH` by its text), then click it.
5. After clicking, wait again — this time for the text element (`By.ID 'finish'`) to actually become visible on the page.
6. Once it's visible, pull out the text with `.text` and print it so it's visible in the console.
7. Assert that the text equals 'Hello World!' and print a pass message if it matches.
8. Close the browser with `driver.quit()` once everything is done.

Implementation (Python / Selenium):

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
driver.maximize_window()

wait = WebDriverWait(driver, 10)

start_button = wait.until(
    EC.element_to_be_clickable((By.XPATH, "//button[text()='Start']"))
)
start_button.click()

text_element = wait.until(
    EC.visibility_of_element_located((By.ID, "finish"))
)

text = text_element.text

print("Dynamic text:", text)

assert text == "Hello World!"

print("Test passed: Dynamic text is displayed correctly.")

driver.quit()
```

Output with Clearly Labelled Screenshots

Screenshot 1: Source Code (Selenium Script)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
driver.maximize_window()

wait = WebDriverWait(driver, 10)

start_button = wait.until(
    EC.element_to_be_clickable((By.XPATH, "//button[text()='Start']"))
)
start_button.click()

text_element = wait.until(
    EC.visibility_of_element_located((By.ID, "finish"))
)

text = text_element.text

print("Dynamic text:", text)

assert text == "Hello World!"

print("Test passed: Dynamic text is displayed correctly.")

driver.quit()
```

Fig. 1 – The script that waits for the Start button to be clickable, clicks it, then waits for the finish text to become visible before reading and asserting it.

Screenshot 2: Execution Log / Console Output

```
PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
; (& "c:\Users\heyas\OneDrive\Desktop\Wipro Selenium\.venv\Scripts\Activate.ps1")
(.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> python Asst_2.py
Dynamic text: Hello World!
Test passed: Dynamic text is displayed correctly.
(.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> █
```

Fig. 2 – Terminal output after running the script — it prints the dynamic text that appeared and confirms the assertion passed.

Result, Observation, and Conclusion

Result: The script waited for the Start button, clicked it, then waited for the text to load and correctly picked up 'Hello World!' once it appeared. The assertion passed and the console printed the expected confirmation message.

Observation: Without an explicit wait, trying to read the finish element right after clicking Start would've failed since the text takes a few seconds to load. Using WebDriverWait with expected_conditions handled that timing gap on its own — the script just waited exactly as long as it needed to, instead of a fixed guess like `time.sleep(5)` which either wastes time or isn't long enough. The terminal output confirms the whole thing ran without any errors.

Conclusion: This one really drove home why explicit waits matter more than fixed delays when dealing with dynamic content. `element_to_be_clickable` and `visibility_of_element_located` both did their job of pausing the script only as long as necessary, making the automation more reliable and less flaky than sleeping for a guessed number of seconds — a small change that makes a script actually usable on real, unpredictable pages.